

SESSION 1 LAB EXP

Nithya sree.S (192511058)

Question 1: First Palindromic String in an Array

Problem: Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "".

Program (C):

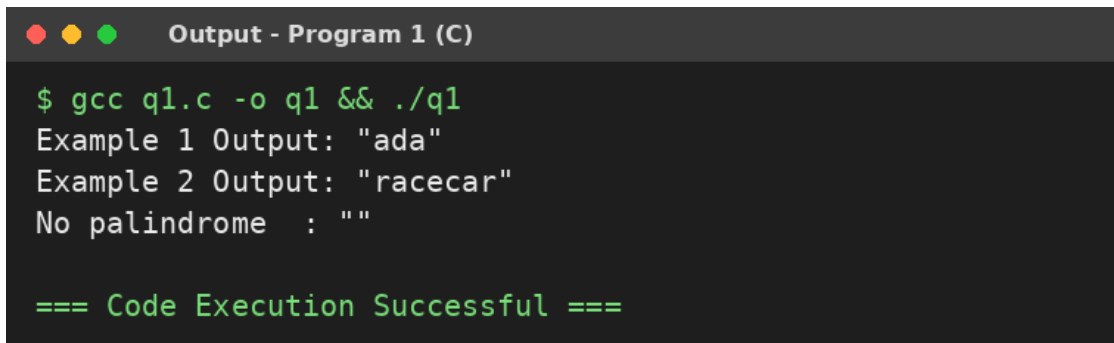
```
#include <stdio.h>
#include <string.h>

int isPalindrome(const char *s) {
    int i = 0, j = (int)strlen(s) - 1;
    while (i < j) {
        if (s[i++] != s[j--]) return 0;
    }
    return 1;
}

const char *firstPalindrome(const char *words[], int n) {
    for (int i = 0; i < n; i++)
        if (isPalindrome(words[i])) return words[i];
    return "";
}

int main() {
    const char *w1[] = {"abc", "car", "ada", "racecar", "cool"};
    const char *w2[] = {"notapalindrome", "racecar"};
    const char *w3[] = {"abc", "def"};
    printf("Example 1 Output: \"%s\\n\", firstPalindrome(w1, 5));
    printf("Example 2 Output: \"%s\\n\", firstPalindrome(w2, 2));
    printf("No palindrome : \"%s\\n\", firstPalindrome(w3, 2));
    return 0;
}
```

Output:



```
Output - Program 1 (C)

$ gcc q1.c -o q1 && ./q1
Example 1 Output: "ada"
Example 2 Output: "racecar"
No palindrome : ""

=== Code Execution Successful ===
```

Complexity: Time: $O(N \times L)$ where L is the average word length. Space: $O(1)$.

Question 2: Count Common Elements Between Two Arrays

Problem: Given integer arrays `nums1` and `nums2` of sizes `n` and `m`, return `[answer1, answer2]`, where `answer1` is the number of indices `i` such that `nums1[i]` exists in `nums2`, and `answer2` is the number of indices `i` such that `nums2[i]` exists in `nums1`.

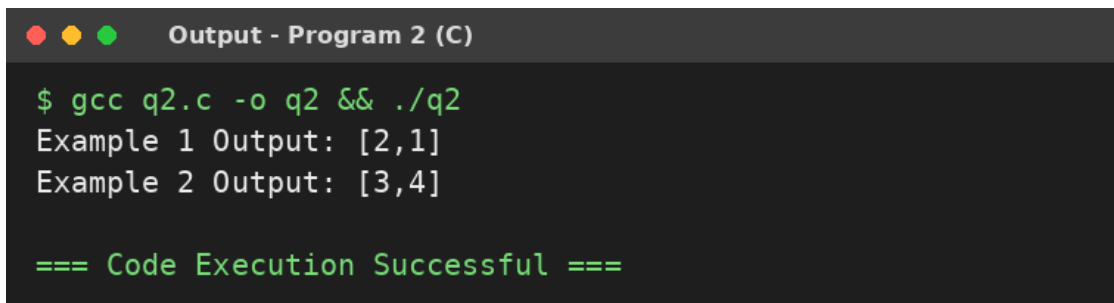
Program (C):

```
#include <stdio.h>
#include <string.h>

/* Values are in range 1..100, so a presence table gives O(n + m) time */
void findIntersectionValues(int nums1[], int n, int nums2[], int m, int ans[2]) {
    int in1[101] = {0}, in2[101] = {0};
    for (int i = 0; i < n; i++) in1[nums1[i]] = 1;
    for (int i = 0; i < m; i++) in2[nums2[i]] = 1;
    ans[0] = ans[1] = 0;
    for (int i = 0; i < n; i++) if (in2[nums1[i]]) ans[0]++;
    for (int i = 0; i < m; i++) if (in1[nums2[i]]) ans[1]++;
}

int main() {
    int a1[] = {2, 3, 2}, b1[] = {1, 2};
    int a2[] = {4, 3, 2, 3, 1}, b2[] = {2, 2, 5, 2, 3, 6};
    int ans[2];
    findIntersectionValues(a1, 3, b1, 2, ans);
    printf("Example 1 Output: [%d,%d]\n", ans[0], ans[1]);
    findIntersectionValues(a2, 5, b2, 6, ans);
    printf("Example 2 Output: [%d,%d]\n", ans[0], ans[1]);
    return 0;
}
```

Output:



```
Output - Program 2 (C)

$ gcc q2.c -o q2 && ./q2
Example 1 Output: [2,1]
Example 2 Output: [3,4]

=== Code Execution Successful ===
```

Complexity: Time: $O(n + m)$. Space: $O(1)$ (fixed-size presence tables for values 1..100).

Question 3: Sum of Squares of Distinct Counts of All Subarrays

Problem: Given a 0-indexed integer array `nums`, return the sum of the squares of the distinct counts of all its subarrays.

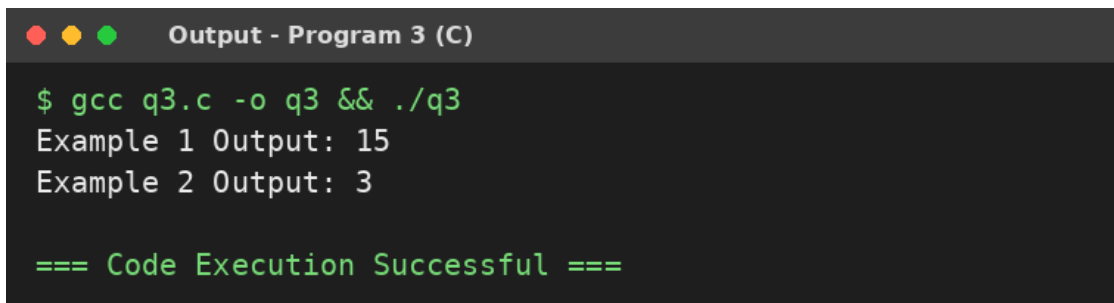
Program (C):

```
#include <stdio.h>
#include <string.h>

/* For every start index i, extend j and keep a running distinct count. */
int sumCounts(int nums[], int n) {
    int total = 0;
    for (int i = 0; i < n; i++) {
        int seen[101];
        memset(seen, 0, sizeof(seen));
        int distinct = 0;
        for (int j = i; j < n; j++) {
            if (!seen[nums[j]]) { seen[nums[j]] = 1; distinct++; }
            total += distinct * distinct;
        }
    }
    return total;
}

int main() {
    int a[] = {1, 2, 1};
    int b[] = {1, 1};
    printf("Example 1 Output: %d\n", sumCounts(a, 3));
    printf("Example 2 Output: %d\n", sumCounts(b, 2));
    return 0;
}
```

Output:



```
Output - Program 3 (C)

$ gcc q3.c -o q3 && ./q3
Example 1 Output: 15
Example 2 Output: 3

=== Code Execution Successful ===
```

Complexity: Time: $O(n^2)$. Space: $O(1)$ (fixed-size table for values 1..100).

Question 4: Count Equal Pairs Whose Index Product is Divisible by k

Problem: Given a 0-indexed integer array `nums` of length `n` and an integer `k`, return the number of pairs (i, j) where $0 \leq i < j < n$, `nums[i] == nums[j]` and $(i * j)$ is divisible by `k`.

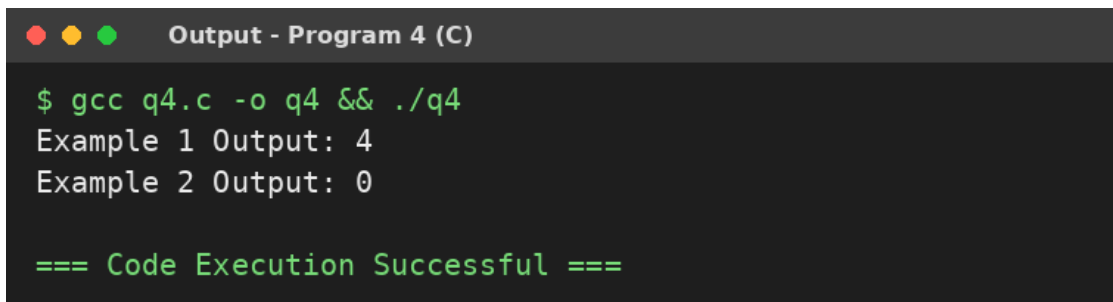
Program (C):

```
#include <stdio.h>

int countPairs(int nums[], int n, int k) {
    int count = 0;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (nums[i] == nums[j] && (i * j) % k == 0)
                count++;
    return count;
}

int main() {
    int a[] = {3, 1, 2, 2, 2, 1, 3};
    int b[] = {1, 2, 3, 4};
    printf("Example 1 Output: %d\n", countPairs(a, 7, 2));
    printf("Example 2 Output: %d\n", countPairs(b, 4, 1));
    return 0;
}
```

Output:



```
Output - Program 4 (C)

$ gcc q4.c -o q4 && ./q4
Example 1 Output: 4
Example 2 Output: 0

=== Code Execution Successful ===
```

Complexity: Time: $O(n^2)$. Space: $O(1)$.

Question 5: Program for the Given Test Cases with Least Time Complexity

Problem: Test cases: {1,2,3,4,5} -> 5, {7,7,7,7,7} -> 7, {-10,2,3,-4,5} -> 5. The expected outputs show the task is to find the maximum element.

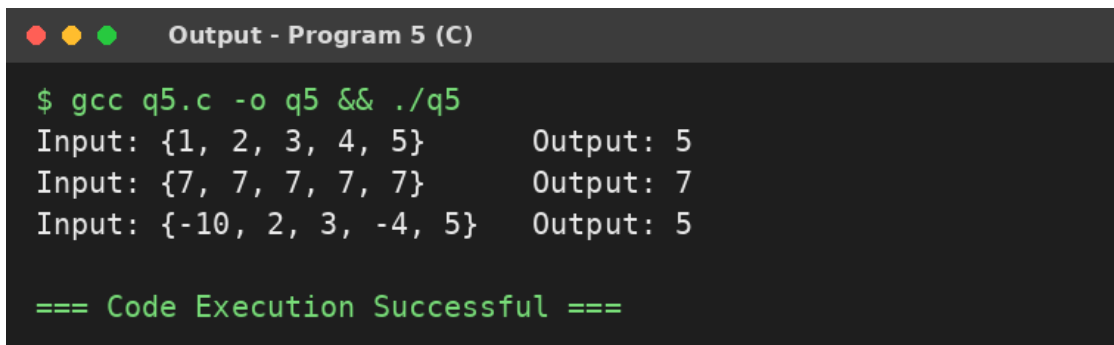
Program (C):

```
#include <stdio.h>

/* Single pass linear scan: O(n) time, O(1) space (the least possible) */
int findMax(int arr[], int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > max) max = arr[i];
    return max;
}

int main() {
    int t1[] = {1, 2, 3, 4, 5};
    int t2[] = {7, 7, 7, 7, 7};
    int t3[] = {-10, 2, 3, -4, 5};
    printf("Input: {1, 2, 3, 4, 5}    Output: %d\n", findMax(t1, 5));
    printf("Input: {7, 7, 7, 7, 7}    Output: %d\n", findMax(t2, 5));
    printf("Input: {-10, 2, 3, -4, 5}  Output: %d\n", findMax(t3, 5));
    return 0;
}
```

Output:



```
Output - Program 5 (C)

$ gcc q5.c -o q5 && ./q5
Input: {1, 2, 3, 4, 5}    Output: 5
Input: {7, 7, 7, 7, 7}    Output: 7
Input: {-10, 2, 3, -4, 5}  Output: 5

=== Code Execution Successful ===
```

Complexity: Time: $O(n)$ - the least possible, since every element must be inspected once. Space: $O(1)$.

Question 6: Sort a List and Find the Maximum Element

Problem: Sort the list using an efficient sorting algorithm, then find the maximum element in the sorted list.
Handle: empty list, single element list, and all elements equal.

Program (C):

```
#include <stdio.h>
#include <stdlib.h>

int cmp(const void *a, const void *b) {
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);
}

/* Sort with qsort (efficient, O(n log n)) then take the last element */
void sortAndFindMax(int arr[], int n) {
    if (n == 0) {
        printf("The list is empty (None)\n");
        return;
    }
    qsort(arr, n, sizeof(int), cmp);
    printf("Sorted list: ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\nMaximum element: %d\n", arr[n - 1]);
}

int main() {
    int t2[] = {5};
    int t3[] = {3, 3, 3, 3, 3};

    printf("Test 1: Empty list []\n");
    sortAndFindMax(NULL, 0);
    printf("\nTest 2: Single element [5]\n");
    sortAndFindMax(t2, 1);
    printf("\nTest 3: All same [3, 3, 3, 3, 3]\n");
    sortAndFindMax(t3, 5);
    return 0;
}
```

Output:

```
Output - Program 6 (C)

$ gcc q6.c -o q6 && ./q6
Test 1: Empty list []
The list is empty (None)

Test 2: Single element [5]
Sorted list: 5
Maximum element: 5

Test 3: All same [3, 3, 3, 3, 3]
Sorted list: 3 3 3 3 3
Maximum element: 3

=== Code Execution Successful ===
```

Complexity: Time: $O(n \log n)$ for sorting; the maximum is read in $O(1)$ from the end. Space: $O(\log n)$ for qsort recursion.

Question 7: Unique Elements from a List (Space Complexity)

Problem: Take a list of n numbers and create a new list containing only the unique elements. State the space complexity of the algorithm.

Program (C):

```
#include <stdio.h>
#include <stdlib.h>

/* Hash set with linear probing: O(n) average time, O(n) extra space */
void uniqueElements(int arr[], int n) {
    int size = 2 * n + 1;
    long *table = malloc(size * sizeof(long));
    char *used = calloc(size, 1);
    printf("[");
    int first = 1;
    for (int i = 0; i < n; i++) {
        unsigned int h = ((unsigned int)arr[i] * 2654435761u) % size;
        int found = 0;
        while (used[h]) {
            if (table[h] == arr[i]) { found = 1; break; }
            h = (h + 1) % size;
        }
        if (!found) {
            used[h] = 1;
            table[h] = arr[i];
            printf("%s%d", first ? "" : ", ", arr[i]);
            first = 0;
        }
    }
    printf("]\n");
    free(table);
    free(used);
}

int main() {
    int a[] = {3, 7, 3, 5, 2, 5, 9, 2};
    int b[] = {-1, 2, -1, 3, 2, -2};
    int c[] = {1000000, 999999, 1000000};
    printf("Test 1 Output: "); uniqueElements(a, 8);
    printf("Test 2 Output: "); uniqueElements(b, 6);
    printf("Test 3 Output: "); uniqueElements(c, 3);
    printf("\nSpace complexity: O(n)\n");
    return 0;
}
```

Output:

```
Output - Program 7 (C)

$ gcc q7.c -o q7 && ./q7
Test 1 Output: [3, 7, 5, 2, 9]
Test 2 Output: [-1, 2, 3, -2]
Test 3 Output: [1000000, 999999]

Space complexity: O(n)

=== Code Execution Successful ===
```

Complexity: Time: $O(n)$ average. Space complexity: $O(n)$ - the hash set and output list may hold up to n distinct elements.

Question 8: Bubble Sort with Big-O Analysis

Problem: Sort an array of integers using the bubble sort technique and analyse its time complexity using Big-O notation.

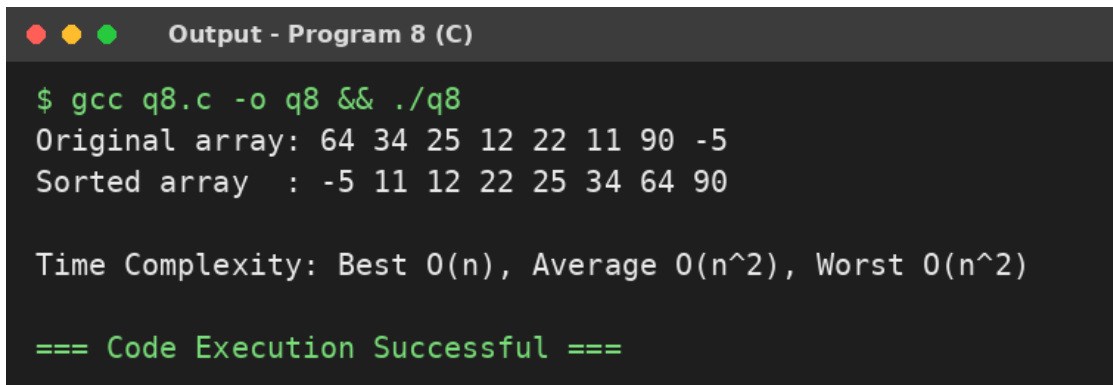
Program (C):

```
#include <stdio.h>

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int t = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = t;
                swapped = 1;
            }
        }
        if (!swapped) break; /* already sorted -> best case O(n) */
    }
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90, -5};
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Original array: ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    bubbleSort(arr, n);
    printf("\nSorted array : ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n\nTime Complexity: Best O(n), Average O(n^2), Worst O(n^2)\n");
    return 0;
}
```

Output:



```
Output - Program 8 (C)

$ gcc q8.c -o q8 && ./q8
Original array: 64 34 25 12 22 11 90 -5
Sorted array  : -5 11 12 22 25 34 64 90

Time Complexity: Best O(n), Average O(n^2), Worst O(n^2)

=== Code Execution Successful ===
```

Complexity: Best case $O(n)$ (already sorted, thanks to the swapped flag), Average $O(n^2)$, Worst $O(n^2)$.
Space: $O(1)$.

Question 9: Binary Search for Element x

Problem: Check whether a given number x exists in an array using binary search and analyse its time complexity. Note: binary search requires sorted data and the sample array X is unsorted, so the program first sorts (value, original position) pairs and then binary-searches. The original 1-based position is reported, which matches the expected output (10 is at position 5).

Program (C):

```
#include <stdio.h>

typedef struct { int value; int pos; } Item;

/* Insertion sort on (value, original position) pairs */
void sortItems(Item a[], int n) {
    for (int i = 1; i < n; i++) {
        Item key = a[i];
        int j = i - 1;
        while (j >= 0 && a[j].value > key.value) { a[j + 1] = a[j]; j--; }
        a[j + 1] = key;
    }
}

/* Binary search: O(log n). Returns original 1-based position or -1 */
int binarySearch(Item a[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid].value == key) return a[mid].pos;
        if (a[mid].value < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

void search(int X[], int n, int key) {
    Item a[20];
    for (int i = 0; i < n; i++) { a[i].value = X[i]; a[i].pos = i + 1; }
    sortItems(a, n);          /* binary search needs sorted data */
    int p = binarySearch(a, n, key);
    if (p != -1) printf("Element %d is found at position %d\n", key, p);
    else      printf("Element %d is not found\n", key);
}

int main() {
    int X[] = {3, 4, 6, -9, 10, 8, 9, 30};
    int n = sizeof(X) / sizeof(X[0]);
    search(X, n, 10);
    search(X, n, 100);
    printf("\nTime Complexity: O(log n) search (best O(1))\n");
    return 0;
}
```

Output:

```
Output - Program 9 (C)

$ gcc q9.c -o q9 && ./q9
Element 10 is found at position 5
Element 100 is not found

Time Complexity:  $O(\log n)$  search (best  $O(1)$ )

=== Code Execution Successful ===
```

Complexity: Binary search: Best $O(1)$, Average/Worst $O(\log n)$. Space: $O(1)$ for the search ($O(n)$ for the position pairs).

Question 10: Sort an Array in $O(n \log n)$ with the Smallest Space Complexity

Problem: Given an array of integers nums, sort it in ascending order without using built-in functions, in $O(n \log n)$ time and with the smallest possible space complexity. Heap sort is used because it is in-place.

Program (C):

```
#include <stdio.h>

/* Heap sort:  $O(n \log n)$  time in every case,  $O(1)$  extra space */
void heapify(int a[], int n, int i) {
    while (1) {
        int largest = i, l = 2 * i + 1, r = 2 * i + 2;
        if (l < n && a[l] > a[largest]) largest = l;
        if (r < n && a[r] > a[largest]) largest = r;
        if (largest == i) break;
        int t = a[i]; a[i] = a[largest]; a[largest] = t;
        i = largest;
    }
}

void sortArray(int a[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) heapify(a, n, i);
    for (int i = n - 1; i > 0; i--) {
        int t = a[0]; a[0] = a[i]; a[i] = t;
        heapify(a, i, 0);
    }
}

void show(const char *label, int a[], int n) {
    printf("%s[", label);
    for (int i = 0; i < n; i++) printf("%s%d", i ? ", " : "", a[i]);
    printf("]\n");
}

int main() {
    int a[] = {5, 2, 3, 1};
    int b[] = {5, 1, 1, 2, 0, 0};
    show("Input : ", a, 4); sortArray(a, 4); show("Output: ", a, 4);
    printf("\n");
    show("Input : ", b, 6); sortArray(b, 6); show("Output: ", b, 6);
    return 0;
}
```

Output:

```
Output - Program 10 (C)

$ gcc q10.c -o q10 && ./q10
Input : [5, 2, 3, 1]
Output: [1, 2, 3, 5]

Input : [5, 1, 1, 2, 0, 0]
Output: [0, 0, 1, 1, 2, 5]

=== Code Execution Successful ===
```

Complexity: Time: $O(n \log n)$ in best, average and worst cases. Space: $O(1)$ - in-place, iterative heapify.